PYTHON · LESSON 2

What will we cover?

We start with your first Python commands, then build up to storing, checking, and converting values.

1. **print() and comments**
   Display output and annotate your code
2. **Variables**
   Store values and name them
3. **Data types**
   str, int, float, bool — and how to check them
4. **Type conversion and input**
   Convert between types and read from the keyboard

Goal: write a program that stores personal information in variables and prints a formatted profile.

PYTHON BASICS

PRINT()

`print()` displays something on the screen. You will use it in almost every program.

WHAT IT DOES

Displays output

Whatever you put inside the parentheses appears on the screen when the program runs. Python processes each `print()` line in order, top to bottom.

You can print text, numbers, variables, or a mix of all three.

SYNTAX

Text needs quotes

```
print("Hello, World!")
print("My name is Sara")
print(42)
print(3.14)
print("I am", 25, "years old")
```

> Use **double** quotes `"..."` or **single** quotes `'...'` around text — both work. Numbers go in without quotes.

PYTHON BASICS

print() in action

```python
print("Hello, World!")
print("My name is Sara")
print("I am", 25, "years old")

# Print a blank line
print()

# Numbers — no quotes needed
print(42)
print(3.14)
```

OUTPUT

```
Hello, World!
My name is Sara
I am 25 years old

42
3.14
```

Each `print()` starts on a new line. A `print()` with nothing inside prints a blank line.

Commas inside `print()` add a space between the values automatically.

PYTHON BASICS

Comments

A comment is a note written for the programmer. Python ignores everything after a `#` symbol on that line.

WHY USE THEM

Explain your thinking

Comments do not affect what the program does. They help you and others understand *why* the code is written the way it is.

Professional code has comments. Get in the habit early.

SYNTAX

Start with

```
# This whole line is a comment
print("Hello!")   # comment at end of line

# Describe what the next line does
print("My name is Sara")

# You can comment out code to disable it
# print("This line will not run")
```

VARIABLES

What is a variable?

A **variable** is a named box that stores a value. You give it a name, and Python remembers the value for you.

WITHOUT A VARIABLE

```
print("Sara")
print("Sara")
print("Sara")
```

You repeat the value every time. If it changes, you must update every single occurrence.

WITH A VARIABLE

```
name = "Sara"

print(name)
print(name)
print(name)
```

Change the value in one place — it updates everywhere automatically.

> Think of a variable as a sticky note: you write a label on it and stick a value underneath. Any time you need the value, just use the label.

VARIABLES

Creating variables

Write the name, then `=`, then the value. Python creates the variable and remembers the value.

EXAMPLES

All four types

```
name       = "Sara"    # text
age        = 25        # whole number
height     = 1.68      # decimal number
is_student = True      # True or False

print(age)             # 25

# Update a value any time
age = 26
print(age)             # 26
```

NAMING RULES

Valid names

✓ Use letters, digits, underscores

✓ Start with a letter or underscore

✓ Use `snake_case` for readability

✕ Cannot start with a digit — `1name`

✕ No spaces — use `my_name` not `my name`

✕ Case matters — `name` ≠ `Name`

DATA TYPES

The four basic types

Every value in Python has a **type**. The type tells Python what the value is and what you can do with it.

STR

String

Text — always inside quotes.

```
name  = "Sara"
city  = 'Berlin'
email = "sara@redi.de"
```

INT

Integer

Whole numbers — no quotes, no decimal point.

```
age      = 25
students = 12
year     = 2026
```

FLOAT

Float

Decimal numbers — no quotes.

```
height  = 1.68
price   = 2.99
average = 8.5
```

BOOL

Boolean

Only two values — must be capitalised.

```
is_student  = True
has_ticket  = False
is_raining  = True
```

DATA TYPES

type() and len()

```
name   = "Sara"
age    = 25
height = 1.68
flag   = True

# type() tells you the type of a value
print(type(name))    # <class 'str'>
print(type(age))     # <class 'int'>
print(type(height))  # <class 'float'>
print(type(flag))    # <class 'bool'>

# len() counts characters in a string
print(len(name))     # 4
print(len("Berlin")) # 6
print(len(""))       # 0  — empty string
```

WHY USE THEM

type()

Use it when you are unsure what type a variable holds — especially useful when debugging.

len()

Counts how many characters are in a string. Works on lists too (you will see that later).

> Both are **built-in functions** — they come with Python and are always available.

TYPE CONVERSION

Converting between types

```
# str → int
age = int("25")
print(age + 1)          # 26

# str → float
price = float("9.99")
print(price * 2)        # 19.98

# int/float → str  (needed when joining text)
age  = 25
print("I am " + str(age) + " years old")

# int → float and back
x = float(5)    # 5.0
y = int(3.9)    # 3  ← decimal is cut, not rounded!
```

THE THREE CONVERTERS

`int()`

Converts to a whole number. Cuts off any decimal — does not round.

`float()`

Converts to a decimal number.

`str()`

Converts anything to text. Required before joining a number into a string with `+`.

TYPE CONVERSION

Common type errors

TYPEERROR

Cannot mix str and int

```
# ✗ This crashes
age = 25
print("I am " + age)
```

Fix: convert with `str(age)` or use a comma instead of `+` :

```
print("I am " + str(age))  # ✓
print("I am", age)          # ✓ comma works too
```

VALUEERROR

Cannot convert non-numbers

```
# ✗ This crashes
age = int("twenty five")
```

Fix: `int()` and `float()` only work on numeric strings:

```
age = int("25")     # ✓  "25" is a valid number
```

> **TypeError** — check your types and add a conversion. **ValueError** — check what you are trying to convert.

USER INPUT

input()

```
# Ask for text — always returns a string
name = input("What is your name? ")
print("Hello, " + name + "!")

# Ask for a number — must convert!
age = int(input("How old are you? "))
print("Next year you will be", age + 1)

# Ask for a decimal number
price = float(input("Enter a price: "))
print("Double:", price * 2)
```

WHAT HAPPENS

```
What is your name? Sara
Hello, Sara!
How old are you? 25
Next year you will be 26
Enter a price: 4.99
Double: 9.98
```

> `input()` always returns a string — even when the user types a number. Wrap it with `int()` or `float()` if you need to do maths.

EXERCISE

Build an About Me program

1. **Create variables**
   Name five variables about yourself: `name`, `age`, `city`, `language`, `is_student`.

2. **Print each one**
   Use `print()` to display every variable on its own line.

3. **Check the types**
   Use `type()` to print the type of each variable.

4. **Update your age**
   Change `age` to next year's value and print it again.

5. **Measure your name**
   Use `len(name)` to print how many letters your name has.

Done when: your output shows all five values, five types, the updated age, and the name length.

```
# Lesson 2 — About Me

# 1. Create your variables
name       =
age        =
city       =
language   =
is_student =

# 2. Print each variable


# 3. Check types with type()


# 4. Update age and print again


# 5. Print the length of your name
```

SOLUTION · ABOUT ME

About Me — solution

```
# 1. Variables
name       = "Sara"
age        = 25
city       = "Berlin"
language   = "Arabic"
is_student = True

# 2. Print each one
print(name)           # Sara
print(age)            # 25
print(city)           # Berlin
print(language)       # Arabic
print(is_student)     # True
```

```
# 3. Types
print(type(name))     # <class 'str'>
print(type(age))      # <class 'int'>
print(type(is_student)) # <class 'bool'>

# 4. Update age
age = 26
print(age)            # 26

# 5. Name length
print(len(name))      # 4
```

TAKE-HOME EXERCISE · VARIABLES

Type detective

For each code snippet, predict the output before you run it. Then run the code to check.

1. What does `type("42")` return?
2. What does `int(3.9)` return?
3. What does `len("ReDI")` return?
4. Does `"I am " + 25` work? Why or why not?
5. What does `str(True)` return?
6. What does `type(3.0)` return?

SOLUTION · TYPE DETECTIVE

Type detective — solution

1. **type("42")**
   `<class 'str'>` — quotes make it a string, not a number.

2. **int(3.9)**
   `3` — int() cuts the decimal off; it does not round up.

3. **len("ReDI")**
   `4` — four characters.

4. **"I am " + 25**
   No — TypeError. Use `str(25)` first, or a comma.

5. **str(True)**
   `"True"` — the boolean becomes the string "True".

6. **type(3.0)**
   `<class 'float'>` — the decimal point makes it a float.

HOMEWORK · LESSON 2

For next class

1. **All About Me** — create variables for `name`, `age`, `city`, `favourite_food`, `favourite_colour`, `is_working`. Print each on its own line.

2. **Type inspector** — for every variable above, print its type with `type()`.

3. **Receipt** — create `item`, `price` (float), `quantity` (int). Print: `Bread – quantity: 2 – price: 1.5`

4. **len() explorer** — create three string variables (name, city, food). Print the length of each. Which is longest?

5. **Mad libs** — create `adjective`, `animal`, `place`, `number`. Print: *The [adjective] [animal] visited [number] places in [place].*

6. **Converter chain** — start with `age_text = "28"`. Convert it to `int`, then to `float`, then back to `str`. Print the type after each step.

7. **Interactive profile** — use `input()` to ask for a name and age. Print a sentence using both. Bonus: print how many letters the name has.

LESSON COMPLETE

You have variables

1. print() and comments
2. Variables
3. str · int · float · bool
4. type() · len() · input()

Next lesson: arithmetic, order of operations, and building a calculator with user input.